

searching should be a read-only operation; hence we pass each matching checkin object through a generic clone() operation.

```

• var allCheckins = JSON.parse(localStorage["checkins"]);
• var matchingCheckins = [];
• allCheckins.forEach(function(checkin) {
•   if (checkin.mood == moodQuery) {
•     matchingCheckins.push(clone(checkin));
•   }
• });
• handler(matchingCheckins);

```

Providing Offline Experiences

It's becoming increasingly important for web-based applications to be accessible offline. Yes, all browsers have caching mechanisms, but they're unreliable and don't always work as you might expect. HTML5 addresses some of the annoyances of being offline with the ApplicationCache interface.

Using the cache interface gives your application three advantages:

- ▶ Offline browsing - users can navigate your full site when they're offline
- ▶ Speed - cached resources are local, and therefore load faster.
- ▶ Reduced server load - the browser will only download resources from the server that have changed.

The Application Cache (or AppCache) allows a developer to specify which files the browser should cache and make available to offline users. Your app will load and work correctly, even if the user presses the refresh button while they're offline.

Using Application Cache

To enable the application cache for an app, include the manifest attribute on the document's html tag:

```

• <html manifest="example.appcache">
•   ...
• </html>

```

The manifest attribute should be included on every page of your web application that you want cached. The browser does not cache a page if it does not contain the manifest attribute (unless it is explicitly listed in the manifest file itself. This means that any page the user navigates to that